

# Tema 8.2: Introducción a GitHub — Colaboración, Control de Versiones y Flujos Asistidos

---

## Resumen

---

En esta lección, usted estudiará las bases de **Git** y **GitHub** orientadas al trabajo colaborativo en ingeniería de software. Aprenderá las mejores prácticas para inicializar repositorios, gestionar ramas de trabajo (*branches*), estructurar confirmaciones (*commits*) explicativos y resolver conflictos de fusión. Asimismo, conocerá cómo interactuar con el control de versiones a través de la interfaz gráfica de **Visual Studio Code** y la asistencia automática de **Google Antigravity**.

---

## Concepto Principal (El 'Qué')

---

El control de versiones es una infraestructura crítica en el desarrollo de software moderno. Se divide en dos componentes tecnológicos que usted debe distinguir: 1. **Git**: Es un sistema de control de versiones distribuido de código abierto que se ejecuta localmente en su computadora. Registra el historial de cambios del código fuente a lo largo del tiempo, permitiendo revertir archivos a estados anteriores y trabajar en paralelo. 2. **GitHub**: Es una plataforma de alojamiento en la nube que almacena repositorios Git remotos. Proporciona herramientas adicionales para la colaboración en equipo, tales como solicitudes de cambio (*Pull Requests*), seguimiento de incidentes (*Issues*), automatización (GitHub Actions) y control de accesos.

Flujo de Trabajo Estándar de Git:

```
[ Directorio de Trabajo ] --( git add )--> [ Área de Preparación ] --( git commit )--> [ Repositorio ]
```

## Componentes Clave de la Colaboración en GitHub

### 1. Gestión de Ramas (Feature Branching)

Una **rama** representa una línea independiente de desarrollo. La rama principal (**main**) siempre debe contener código estable y listo para producción. Cada nueva característica, corrección de error o laboratorio se desarrolla en una rama temporal creada para ese propósito específico. Esto evita que cambios incompletos afecten el trabajo del resto del equipo.

## 2. Solicitudes de Fusión (Pull Requests - PR)

Cuando un desarrollador completa el trabajo en una rama temporal, crea un **Pull Request** en GitHub para solicitar que sus cambios sean incorporados a la rama **main**. El PR es un espacio de discusión técnica donde otros miembros del equipo (o el profesor) revisan el código, hacen sugerencias y aprueban la integración una vez asegurada la calidad.

## 3. Conflictos de Fusión (Merge Conflicts)

Un conflicto ocurre cuando Git no puede fusionar automáticamente dos ramas. Esto sucede comúnmente cuando dos desarrolladores modifican las mismas líneas de un mismo archivo en diferentes ramas y uno de ellos intenta fusionar después de que el otro ya lo ha hecho. Git detiene la fusión y marca el archivo para que el desarrollador decida manualmente qué versión conservar.

---

## Analogía (La Intuición)

Imagine que el desarrollo del proyecto de software es la redacción de una **enciclopedia técnica**: \*

- \* **Git local es su cuaderno de borradores personal**: Usted escribe páginas, tacha párrafos y mantiene un registro de lo que ha hecho en su escritorio.
- \* **GitHub es la editorial central en la nube**: Donde se guarda la copia maestra de la enciclopedia a la que todo el mundo tiene acceso.
- \* **Crear una rama es sacar una fotocopia de un capítulo**: Usted se lleva la fotocopia a su casa para trabajar en una sección específica sin alterar la enciclopedia real.
- \* **Un Pull Request es enviar su propuesta de cambio al editor en jefe**: El editor revisa sus páginas fotocopias y editadas, hace correcciones y, tras dar el visto bueno, pega sus hojas nuevas en el libro oficial.
- \* **Un conflicto de fusión es que dos autores intenten pegar cambios en el mismo párrafo a la vez**: El editor jefe detiene la imprenta y les pide que se sienten a decidir juntos cómo debe quedar redactado finalmente ese párrafo.

---

## Ejemplo de Código e Implementación (El 'Cómo')

A continuación se detalla la secuencia de comandos esenciales en la consola de comandos para un flujo de trabajo colaborativo básico:

```
# 1. Clonar el repositorio remoto a su máquina local
git clone https://github.com/usuario/nombre-del-repositorio.git
cd nombre-del-repositorio

# 2. Crear una rama de desarrollo para su tarea o laboratorio
git checkout -b feature/laboratorio-warmup

# 3. Realizar modificaciones en el código fuente (ej. editar Main.java)

# 4. Consultar el estado de los archivos modificados
git status

# 5. Agregar las modificaciones al área de preparación (staging)
git add src/laboratorio0/Main.java

# 6. Registrar los cambios en el historial local con un mensaje claro
git commit -m "feat: implementar algoritmo de deteccion de fraude financiero"

# 7. Subir la rama temporal a GitHub para iniciar el Pull Request
git push -u origin feature/laboratorio-warmup
```

## Integración en Herramientas de Desarrollo

### 1. Uso en Visual Studio Code (VS Code)

VS Code cuenta con una interfaz gráfica potente e integrada para gestionar Git: - [ ] **Vista de Source Control:** Accesible desde el icono de bifurcación en la barra de herramientas izquierda. Muestra la lista de archivos modificados. - [ ] **Realizar acciones sin comandos:** Puede dar clic en el botón **+** junto a los archivos para hacer `git add`, escribir el mensaje en la caja superior y presionar el botón **Commit**, y luego dar clic en **Sync Changes** para hacer `push`. - [ ] **Resolución gráfica de conflictos:** Cuando ocurre un conflicto, VS Code resalta visualmente las líneas en disputa con colores diferentes y ofrece botones directos para *"Accept Current Change"* (conservar sus cambios locales), *"Accept Incoming Change"* (conservar los cambios que vienen de GitHub) o *"Accept Both Changes"*.

### 2. Uso en el entorno Google Antigravity (AI Agent)

El asistente basado en inteligencia artificial **Google Antigravity** eleva la automatización del control de versiones: - [ ] **Redacción inteligente de commits:** Si usted le pide al agente que registre los

cambios, la IA analizará las diferencias exactas de código (*git diff*) y redactará un mensaje de commit estructurado bajo estándares como *Conventional Commits* (ej. `feat: ...`, `fix: ...`), detallando con rigor qué cambió y por qué. - [ ] **Resolución inteligente de conflictos:** Ante un conflicto de fusión, usted puede delegar la resolución a la IA. Google Antigravity leerá ambas versiones en disputa, comprenderá la lógica funcional de cada cambio, mantendrá las dependencias correctas para evitar fallas de compilación, y aplicará la resolución óptima y segura. - [ ] **Automatización de flujos:** El agente puede encargarse de automatizar tareas complejas como: *"Cree una rama para este bug, arréglole, ejecute los tests de Maven y, si pasan, haga commit y push a la rama remota"*.

---

## Trampas Comunes (Gotchas)

---

1. **Trabajar y confirmar directamente en la rama `main`:** Es una mala práctica que compromete la estabilidad del proyecto. Si usted introduce un error en `main`, bloqueará el trabajo de todo su equipo de forma inmediata.
  2. **Solución:** Acostúmbrese a crear ramas para todo cambio (`git checkout -b nombre-rama`) y use Pull Requests para integrar el código estable.
  3. **Subir archivos pesados o sensibles (Credenciales y dependencias):** Subir la carpeta de dependencias locales (como `target/` en Maven o `.settings/`) o archivos de bases de datos locales hace el repositorio excesivamente pesado. Peor aún, subir archivos de propiedades que contienen llaves de APIs o contraseñas de bases de datos expone la seguridad del sistema en GitHub de forma pública.
  4. **Solución:** Configure de manera obligatoria el archivo `.gitignore` en la raíz del proyecto para indicarle a Git qué directorios y archivos debe ignorar por completo.
- 

## Pregunta de Reflexión

---

Si usted se encuentra en una rama local llamada `feature/cambios` y desea sincronizarla con la versión más reciente de `main` que acaba de ser actualizada en GitHub por su compañero de equipo, ¿cuál es el orden de comandos de Git que debería ejecutar para actualizar su rama local de manera segura y resolver posibles conflictos?